

Ex of LinkedHashMap maintaining the access order (order would least frequently used to highly frequently used at top)

```
public class LinkedHashMapExample {  
    public static void main (String args[]) {
```

```
        System.out.println("LinkedHashMap output");
```

```
        Map<Integer, String> map = new
```

```
            LinkedHashMap(16, 0.75F, true);
```

initial capacity

load factor

access order by default false

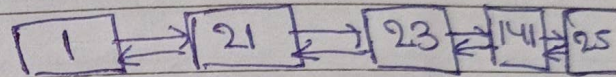
```
        map.put(1, "A");
```

```
        map.put(21, "B");
```

```
        map.put(23, "C");
```

```
        map.put(141, "D");
```

```
        map.put(25, "E");
```

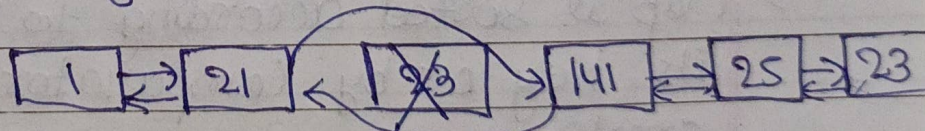


```
// accessing some data
```

```
map.get(23);
```

```
map.forEach((Integer key, String val) ->  
    System.out.println(key + " : " + val);
```

```
}  
}
```



output: LinkedHashMap output is:

1 : A

21 : B

141 : D

25 : E

23 : C

least freq used

highest freq used (Just accessed)

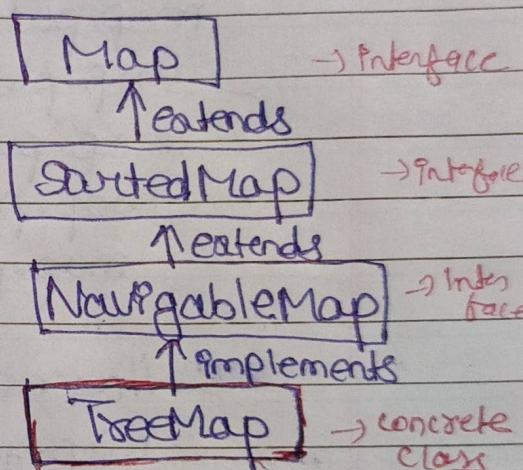
→ worst: $O(n)$
 $O(\log n)$

- Time Complexity is same as of HashMap: Avg $O(1)$
- It is not thread safe and there is no thread safe version available for this.
So we have to explicitly make it this collection thread safe like this:

```
Map<Integer, String> map1 = Collections.synchronizedMap(
    new LinkedHashMap<>());
```

##

< TreeMap >



- Map is sorted according to its natural ordering of its key or by comparator provided during map creation.
- It is based on Red-Black-tree (Self Balancing Binary ~~Tree~~ Search Tree)
- $O(\log n)$ time complexity of insert, remove, get operations.


```
public class TreeMapExample {
    public static void main() {
        Map<Integer, String> map1 = new TreeMap<>();
        (Integer k1, Integer k2) -> k2 - k1;
```

```
        map1.put(21, "SJ");
        map1.put(11, "PJ");
        map1.put(13, "KJ");
        map1.put(5, "TJ");
```

// descending order

```
        map1.forEach((Integer key, String val) ->
            System.out.print(key + " : " + val));
```

```
        System.out.println("-----");
```

```
        Map<Integer, String> map2 = new TreeMap<>();
```

```
        map2.put(21, "SJ");
        map2.put(11, "PJ");
        map2.put(13, "KJ");
        map2.put(5, "TJ");
```

// ascending order

```
        map2.forEach((Integer key, String val) ->
            System.out.print(key + " : " + val));
```

```
    }
```

```
}
```

Output:

21 : SJ

13 : KJ

11 : PJ

5 : TJ

5 : TJ

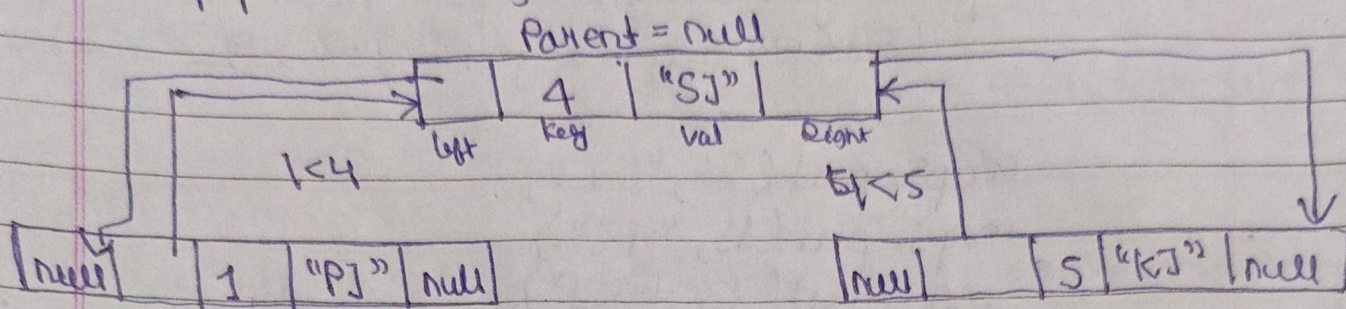
11 : PJ

13 : KJ

21 : SJ


```
map.put(4, "SJ");
map.put(1, "PJ");
map.put(5, "KJ");
```

Left	Parent	Key	Value	Right
------	--------	-----	-------	-------



Internal Structure of TreeMap

S.No

Method Name

Usage

- | | | |
|----|---|--|
| 1. | <pre>Map.Entry<k,v> lowerEntry(k key);</pre> | <pre>map.lowerEntry(23) // 21:B</pre> <p>returns the Entry<key, value> both node less than 23. If the no value returns null.</p> |
| 2. | <pre>k lowerkey(k key);</pre> | <pre>map.lowerkey(23). // 21</pre> <p>- returns the key only which is less than 23. If there is no value return null.</p> |
| 3. | <pre>Map.Entry<k,v> floorEntry (k key);</pre> | <pre>map.floorEntry(24) // 23:C map.floorEntry(23) // 23:C</pre> <p>- returns the Entry(key, value both) which is less or equal to key. If there is no value returns null.</p> |
| 4. | <pre>k floorkey(k key);</pre> | <pre>map.floorkey(24). // 23 map.floorkey(23) // 23</pre> <p>- returns the key only which is less or equal than key. If there is no value returns null.</p> |

5. Map.Entry<k,v> ceilingEntry
(k key);

map.ceilingEntry(23) // 23, C
map.ceilingEntry(24) // 25, E

- returns the Entry (key, value both) which is greater or equal than key. If there is no value returns null.

6. k ceilingKey(k key);

map.ceiling(23) // 23

map.ceiling(24) // 25

- returns the key only which is greater or equal than key. If there is no value returns null.

7. Map.Entry<k,v> higherEntry
(k key);

map.higherEntry(23) // 25, E

map.higherEntry(25) // 41, D

- returns the Entry (key, value both) which is greater than key. If there is no value returns null.

8. k higherKey(k key);

map.higherKey(23) // 25

map.higherKey(25) // 41

- returns the key only which is greater than key. If there is no value returns null.

9. Map.Entry<k,v> firstEntry()

returns the least entry in Map

map.firstEntry() // 1, A

10. lastEntry()

- returns the least entry in Map

map.lastEntry() // 41, D.

11. pollFirstEntry()

remove and return least element from the map

map.pollFirstEntry() // 1, A

12. pollLastEntry()

remove and return the least element from the map

map.pollLastEntry() // 41, A.

key val
 $\{1:2, 2:5, 5:1\} \xRightarrow{\text{reverse}} \{5:2, 2:1, 1:2\}$

13. `NavigableMap<K,V>`
`descendingMap();`

reverse the map and returns

14. `NavigableSet<K>`
`navigableKeySet();`

- [1, 21, 23, 25, 141]

- If TreeMap is sorted in ascending, keys will come in ascending, else in descending.

[141, 25, 23, 21, 1]

15. `NavigableSet<K>`
`descendingKeySet();`

- If TreeMap is sorted in ascending, keys will come in descending else in ascending.

16. `NavigableMap<K,V> headMap`
`(K tokey, boolean inclusive);`

- `map.headMap(23, true);`
 $\{1=A, 21=B, 23=C\}$

17. `NavigableMap<K,V> tailMap`
`(K fromkey, boolean inclusive);`

- `map.tailMap(23, true);`
 $\{23=C, 25=E, 141=D\}$